

Lab 5

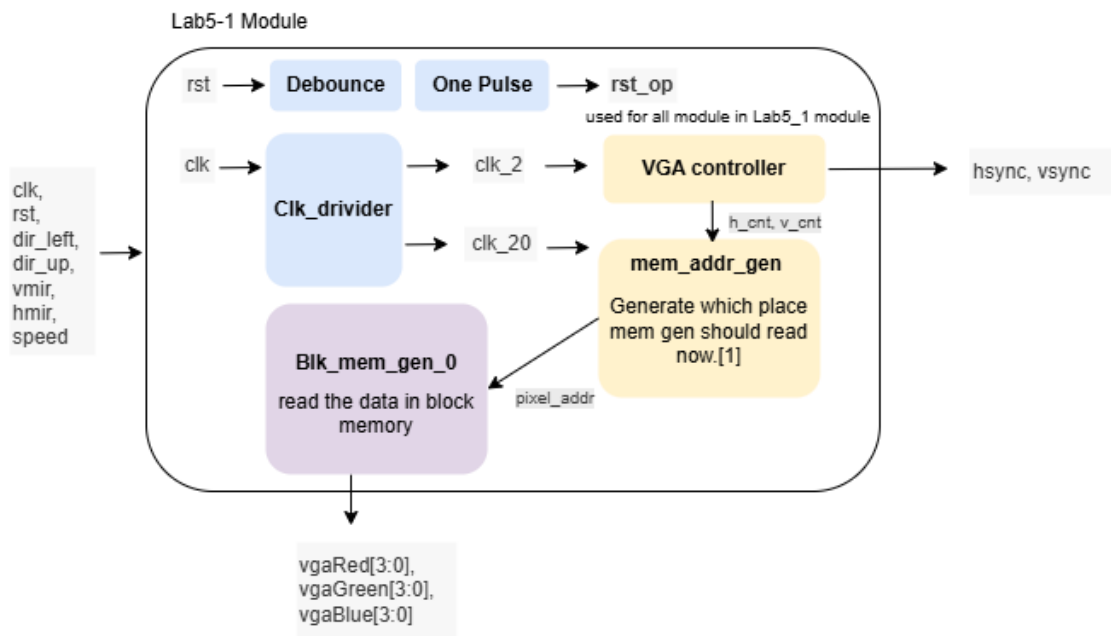
學號: 111062118

姓名: 江佩霖

A. Lab Implementation

1. Lab5_1

(1) Block Diagram



(2) Module / Code Detail

i. vmir/ hmir

```
wire [9:0] h_cnt_new = h_cnt >> 1; // transfer to picture addr
wire [9:0] v_cnt_new = v_cnt >> 1; // vga 640*480 => pic 320*240
wire [9:0] x_res = hmir ? (IMAGE_WIDTH - 1 - h_cnt_new) : h_cnt_new;
wire [9:0] y_res = vmir ? (IMAGE_HEIGHT - 1 - v_cnt_new) : v_cnt_new;
```

用公式計算反轉後應該顯示的圖片座標。

ii. dir_left / dir_up

```
reg [9:0] x_off, y_off; // compute offset for display
// -----
//skip code here
// -----
if (dir_left) begin
    if (hmir) x_off <= (x_off == 0) ? IMAGE_WIDTH - 1 : x_off - 1;
    else x_off <= (x_off == IMAGE_WIDTH - 1) ? 0 : x_off + 1;
end
if (dir_up) begin
    if (vmir) y_off <= (y_off == 0) ? IMAGE_HEIGHT - 1 : y_off - 1;
    else y_off <= (y_off == IMAGE_HEIGHT - 1) ? 0 : y_off + 1;
end
```

用時脈更新當前的 `offset`，使用“位置 + 偏移量”來決定最後要讀取的值。

*由於 `hmir` 以及 `vmir` 是直接對讀取位置反轉，因此 `offset` 在有反轉時計算方式也

要調整，避免向左滾動因為反轉訊號而變成向右滾動。

iii. speed

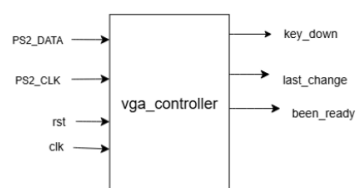
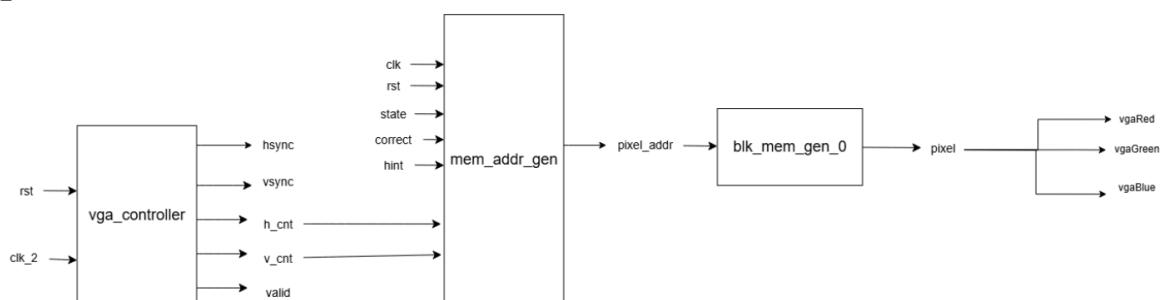
使用 `clk_divider` 產生較快的 `clk` 作為 `speed` 拉起時使用，然後在 `mem_gen` 中設置一個 `counter`，若 `speed` 未拉起，則固定 `count` 4 個 `clk` 才更新一次讀取位置，以達到慢速以及快速捲動。

*由於慢速 `clk` 指定要 2^{22} ，因此較快的 `clk` 我是 generate 2^{20} 然後 `counter` 就設置 4 次更新一次。

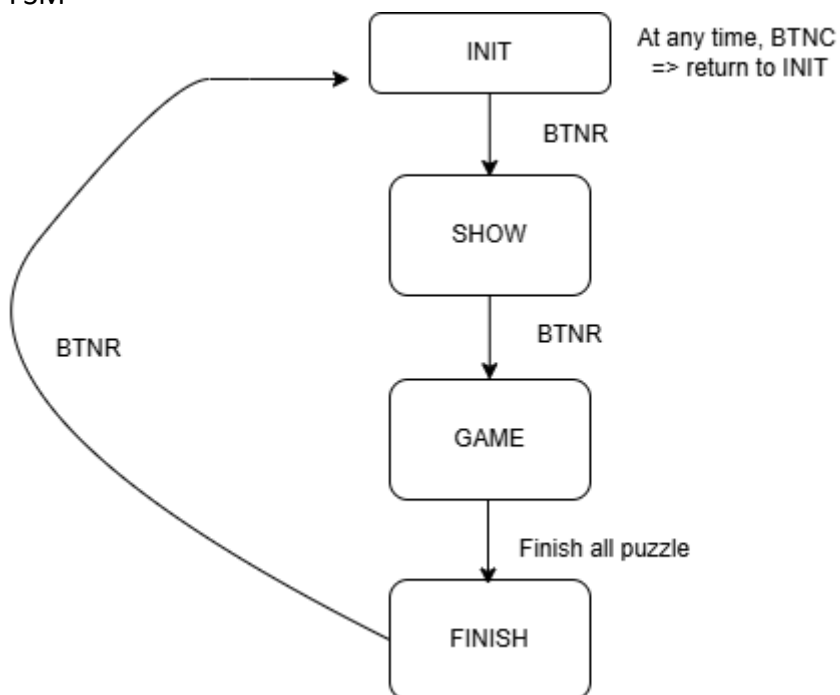
```
// mem_addr_gen module
reg [2:0] count_4; // small divider for slow speed
always @(posedge clk or posedge rst) begin
    if (rst) begin
        count_4 <= 0;
    end else begin
        if (speed == 0) begin
            if (count_4 >= 3'd3) begin
                count_4 <= 0;
                // slow mode → update when counter = 4 <skip code here>
            end else begin
                count_4 <= count_4 + 1;
            end
        end else begin
            // fast mode → update every cycle <skip code here>
        end
    end
end
```

```
// main module
clock_divider #(.n(20)) clk_div_20(
    .clk(clk),
    .clk_div(clk_20)
);
mem_addr_gen mem_inst(
    .clk(clk_20), // input 2^20 clk for speed
    .rst(rst),
    .dir_left(dir_left),
    .dir_up(dir_up),
    .vmir(vmir),
    .hmir(hmir),
    .speed(speed),
    .h_cnt(h_cnt),
    .v_cnt(v_cnt),
    .pixel_addr(pixel_addr)
);
```

1. Lab5_2

(1) Block Diagram
5_2

(2) FSM



(3) Module / Code Detail

i. puzzle 表示& enable 訊號(是否要負片表示)

```
//=====//
//   Puzzle Control
//=====//
// puzzle answer
// TODO: player_input
reg [15:0] player_input;
wire [31:0] puzzle_one_ans  = 32'b1000_1101_1101_1000_0010_0111_0111_0010;
wire [31:0] puzzle_two_ans  = 32'b0100_1110_1011_0100_0001_1011_1110_0001;
wire [31:0] puzzle_three_ans = 32'b0010_0111_1101_1000_0111_0010_1000_1101;
assign puzzle_solved = (player_input == 16'hFFFF) ? 1'b1 : 1'b0;

// puzzle enable : sent to kb for input control
wire [15:0] puzzle_one_en = 16'b0110_0001_0001_1111;
wire [15:0] puzzle_two_en = 16'b0111_1001_0010_0110;
wire [15:0] puzzle_three_en = 16'b0100_1011_1000_1110;
reg [15:0] chosen_puzzle_en;
reg [31:0] chosen_puzzle;
```

直接寫死在 main module

ii. 螢幕顯示邏輯

```
always @(*) begin
    case(state)
        2'd0: pixel = init_pixel; // INIT
        2'd1: begin // SHOW
            // first layer normal
            if(v_cnt < 120) begin
                RGB_reversed = 0;
                if(h_cnt < 160) show_pixel_s = pixel_0;
                else if(h_cnt < 320) show_pixel_s = pixel_1;
                else if(h_cnt < 480) show_pixel_s = pixel_2;
                else show_pixel_s = pixel_3;
            end
            // second layer reversed
            else if(v_cnt < 240) begin
                RGB_reversed = 1;
                if(h_cnt < 160) show_pixel_s = pixel_0;
                else if(h_cnt < 320) show_pixel_s = pixel_1;
                else if(h_cnt < 480) show_pixel_s = pixel_2;
                else show_pixel_s = pixel_3;
            end
            else show_pixel_s = 12'd0;

            // RGB reverse
            if(RGB_reversed) begin
                show_pixel[11:8] = 15 - show_pixel_s[11:8];
                show_pixel[7:4]  = 15 - show_pixel_s[7:4];
                show_pixel[3:0]  = 15 - show_pixel_s[3:0];
            end
            else show_pixel = show_pixel_s;

            pixel = show_pixel;
        end
    end
```

Show: 用位置來計算要顯示哪張圖片，RGB_reversed 表示該圖片要負片顯示

```

2'd2, 2'd3: begin // GAME / FINISH
    idx = 15 - ((v_cnt/120)*4 + (h_cnt/160));
    if(puzzle_placed[idx]) begin
        RGB_reversed = puzzle_is_clue[idx];
        case(idx) ...
        endcase

        if(RGB_reversed) begin
            game_pixel[11:8] = 15 - game_pixel_s[11:8];
            game_pixel[7:4] = 15 - game_pixel_s[7:4];
            game_pixel[3:0] = 15 - game_pixel_s[3:0];
        end
        else game_pixel = game_pixel_s;

        pixel = game_pixel;
    end
    else pixel = 12'd0;
end

```

Case(idx)負責判斷每個位置的 player input 選擇的圖片，並且顯示於該位置。
顯示前需要檢查兩個部分

1. 該位置是否是 puzzle 的一部分(透過寫死的 enable 檢查)
2. 該位置是否已經鎖定並且正在填寫答案(用 puzzle_placed 檢查，當使用者按下 enter 確定在位置輸入且該位置非原本 puzzle 一部分時，會將 puzzle_placed 設 1 讓這個 block 顯示圖片)
3. 該位置要輸出正片/負片，同樣可以使用 enable 檢查

iii. 鎖定玩家答案

```

// -----
// update player answer
// -----
always @(posedge clk) begin
    if(state == 2'd2) begin // GAME
        // submit image: lock the current position
        if(submit_img) begin
            now_valid_pos[current_pos] <= 0;
        end

        // submit position: update answer and mark placed
        if(submit_pos && !puzzle_is_clue[current_pos] && now_valid_pos[current_pos]) begin
            case(current_pos)
                0: player_ans[1:0] <= current_img;
                1: player_ans[3:2] <= current_img;
                2: player_ans[5:4] <= current_img;
                3: player_ans[7:6] <= current_img;
                4: player_ans[9:8] <= current_img;
                5: player_ans[11:10] <= current_img;
                6: player_ans[13:12] <= current_img;
                7: player_ans[15:14] <= current_img;
                8: player_ans[17:16] <= current_img;
                9: player_ans[19:18] <= current_img;
                10: player_ans[21:20] <= current_img;
                11: player_ans[23:22] <= current_img;
                12: player_ans[25:24] <= current_img;
                13: player_ans[27:26] <= current_img;
                14: player_ans[29:28] <= current_img;
                15: player_ans[31:30] <= current_img;
            endcase
            puzzle_placed[current_pos] <= 1; // mark as placed
        end
    end
end

```

Keyboard 輸入端會有兩個確認訊號: submit_pos / submit_img，分別是確認輸入位置 enter 以及確認圖片的 enter(keyboard 端會檢查輸入是否是當前可以接受的輸入)。當收

到 submit_img 訊號後，display 會將 keyboard 那邊收到的對應 img 存到對應的 answer 位置中，並把 puzzle_placed 對應的位置鎖起來(輸入端會根據這個訊號判斷輸入的位置是否能被輸入，再傳送確認訊號)。

iv. 圖片 addr 計算:

```
assign pixel_addr = ( (h_cnt) + 160 * (v_cnt)) % 19200;
```

v. led 顯示

```
integer i;

always @(*) begin
    puzzle_solved = 16'h0000;

    if (check_en) begin
        for(i=0; i<16; i=i+1) begin
            if (answer[i*2+:2] == display_puzzle_transform[i*2+:2]) begin
                puzzle_solved[i] = 1'b1;
            end
        end
    end
end
```

透過 puzzle_solved 這個訊號來表示正確與否，依照 i 所述的寫死的 puzzle 來逐一檢查 player_answer 對應位置答案是否相同。

vi. keyboard control

```
//-----
// Input control logic (two-step selection)
//-----
always @(posedge clk or posedge rst) begin
    if (rst) begin...
    end
    else if (keyboard_en && key_valid_op) begin
        submit_img <= 0;
        submit_pos <= 0;

        // --- Step 1: select position (0~15) ---
        if (!pos_locked) begin
            case (key_num) ...
            endcase

            // Lock position only if valid
            if (key_num == 5'd21 && valid_pos[sel_pos] == 1'b0) begin
                pos_locked <= 1;
                submit_pos <= 1;
                current_pos <= sel_pos;
            end
        end
        // --- Step 2: select image (1~4) ---
        else begin
            case (key_num)
                5'd1: sel_img <= 3'd1;
                5'd2: sel_img <= 3'd2;
                5'd3: sel_img <= 3'd3;
                5'd4: sel_img <= 3'd4;
            endcase
            current_img <= sel_img;

            if (key_num == 5'd21) begin
                submit_img <= 1;
                pos_locked <= 0; // unlock for next round
            end
        end
    end
end
```

B. Questions and Discussions

Provide your answer to the Questions and Discussions in the lab assignment.

A.

- (1) image compression：進行圖片壓縮，需要顯示時，再進行解壓。
- (2) tile-based rendering：將圖片分割成小區塊，只儲存獨特的區塊，若出現相同的區塊，就不用重複儲存。
- (3) color depth reduction：減少每個像素的位元數，以節省 BRAM 空間，但可能會降低畫質。

B. 宣告棋盤格 => VGA 依照掃描到的 pixel 座標來取顏色

```
reg [11:0] img [0:15][0:15];  
initial begin  
    integer x, y;  
    for (y = 0; y < 16; y = y + 1) begin  
        for (x = 0; x < 16; x = x + 1) begin  
            img[y][x] = ((x+y) % 2 == 0) ? 12'hFFF : 12'h000;  
        end  
    end  
end  
assign {r, g, b} = img[v_cnt[3:0]][h_cnt[3:0]];
```

C. 運用連續替換照片製造小男孩在移動的視覺效果

- (1) 處理六個動作的照片
- (2) 先顯示第一個動作
- (3) 停止些許 cycle
- (4) Loop 執行
 - i 清除前一張圖片
 - ii 顯示下一張圖片
 - iii 停止些許 cycle

C. Problem Encountered

這次 lab 要用到很多 use 插槽，一開始還想說會不會不夠用，還好學校電腦給力(?)。

由於我的 coding style 是喜歡接很多個 module，讓 main module 主要負責 FSM 的轉換，這次第二個 lab 作業要在 keyboard 輸入以及展示這兩個 module 互動，讓我苦惱滿久的。還好最後有寫出來。

D. Suggestions

希望考試不要出太難(?)。